Go 语言笔试面试题(实现原理)

2020-09-04 23:10:10

golang interview questions

Go 语言笔试面试题汇总 ， Github

Q1 init() 函数是什么时候执行的？

答案

`init()` 函数是 Go 程序初始化的一部分。Go 程序初始化先于 main 函数，由 runtime 初始化每个导入的包，初始化顺序不是按照从上到下的导入顺序，而是按照解析的依赖关系，没有依赖的包最先初始化。

每个包首先初始化包作用域的常量和变量（常量优先于变量），然后执行包的 `init()` 函数。同一个包，甚至是同一个源文件可以有多个 `init()` 函数。`init()` 函数没有入参和返回值，不能被其他函数调用，同一个包内多个 `init()` 函数的执行顺序不作保证。

一句话总结： import –> const –> var –> `init()` –> `main()`

示例：

```
package main

import "fmt"

func init()  {
```

```go
    fmt.Println("init1:", a)
}

func init()  {
    fmt.Println("init2:", a)
}

var a = 10
const b = 100

func main() {
    fmt.Println("main:", a)
}
// 执行结果
// init1: 10
// init2: 10
// main: 10
```

Q2 Go 语言的局部变量分配在栈上还是堆上？

答案

由编译器决定。Go 语言编译器会自动决定把一个变量放在栈还是放在堆，编译器会做逃逸分析(escape analysis)，当发现变量的作用域没有超出函数范围，就可以在栈上，反之则必须分配在堆上。

```go
func foo() *int {
    v := 11
    return &v
}

func main() {
    m := foo()
    println(*m) // 11
}
```

foo() 函数中，如果 v 分配在栈上，foo 函数返回时，&v 就不存在了，但是这段函数是能够正常运行的。Go 编译器发现 v 的引用脱离了 foo 的作用域，会将其分配在堆上。因此，main 函数中仍能够正常访问该值。

Q3 2 个 interface 可以比较吗？

答案

Go 语言中，interface 的内部实现包含了 2 个字段，类型 T 和 值 V，interface 可以使用 == 或 != 比较。2 个 interface 相等有以下 2 种情况

1) 两个 interface 均等于 nil（此时 V 和 T 都处于 unset 状态）
2) 类型 T 相同，且对应的值 V 相等。

看下面的例子：

```go
type Stu struct {
    Name string
}

type StuInt interface{}

func main() {
    var stu1, stu2 StuInt = &Stu{"Tom"}, &Stu{"Tom"}
    var stu3, stu4 StuInt = Stu{"Tom"}, Stu{"Tom"}
    fmt.Println(stu1 == stu2) // false
    fmt.Println(stu3 == stu4) // true
}
```

`stu1` 和 `stu2` 对应的类型是 `*Stu`，值是 Stu 结构体的地址，两个地址不同，因此结果为 false。 `stu3` 和 `stu4` 对应的类型是 `Stu`，值是 Stu 结构体，且各字段相等，因此结果为 true。

Q4 两个 nil 可能不相等吗？

答案

可能。

接口(interface) 是对非接口值(例如指针，struct 等)的封装，内部实现包含 2 个字段，类型 `T` 和 值 `V`。一个接口等于 nil，当且仅当 T 和 V 处于 unset 状态（T=nil，V is unset）。

- 两个接口值比较时，会先比较 T，再比较 V。
- 接口值与非接口值比较时，会先将非接口值尝试转换为接口值，再比较。

```go
func main() {
    var p *int = nil
    var i interface{} = p
    fmt.Println(i == p) // true
    fmt.Println(p == nil) // true
    fmt.Println(i == nil) // false
}
```

上面这个例子中，将一个 nil 非接口值 p 赋值给接口 i，此时，i 的内部字段为 `(T=*int, V=nil)`，i 与 p 作比较时，将 p 转换为接口后再比较，因此 `i == p`，p 与 nil 比较，直接比较值，所以 `p == nil`。

但是当 i 与 nil 比较时，会将 nil 转换为接口 `(T=nil, V=nil)`，与 i `(T=*int, V=nil)` 不相等，因此 `i != nil`。因此 V 为 nil ，但 T 不为 nil 的接口不等于 nil。

Q5 简述 Go 语言 GC(垃圾回收)的工作原理

答案

最常见的垃圾回收算法有标记清除(Mark-Sweep) 和引用计数(Reference Count)，Go 语言采用的是标记清除算法。并在此基础上使用了三色标记法和写屏障技术，提高了效率。

标记清除收集器是跟踪式垃圾收集器，其执行过程可以分成标记（Mark）和清除（Sweep）两个阶段：

- 标记阶段 — 从根对象出发查找并标记堆中所有存活的对象；
- 清除阶段 — 遍历堆中的全部对象，回收未被标记的垃圾对象并将回收的内存加入空闲链表。

标记清除算法的一大问题是在标记期间，需要暂停程序（Stop the world，STW），标记结束之后，用户程序才可以继续执行。为了能够异步执行，减少 STW 的时间，Go 语言采用了三色标记法。

三色标记算法将程序中的对象分成白色、黑色和灰色三类。

- 白色：不确定对象。
- 灰色：存活对象，子对象待处理。
- 黑色：存活对象。

标记开始时，所有对象加入白色集合（这一步需 STW ）。首先将根对象标记为灰色，加入灰色集合，垃圾搜集器取出一个灰色对象，将其标记为黑色，并将其指向的对象标记为灰色，加入灰色集合。重复这个过程，直到灰色集合为空为止，标记阶段结束。那么白色对象即可需要清理的对象，而黑色对象均为根可达的对象，不能被清理。

三色标记法因为多了一个白色的状态来存放不确定对象，所以后续的标记阶段可以并发地执行。当然并发执行的代价是可能会造成一些遗漏，因为那些早先被标记为黑色的对象可能目前已经是不可达的了。所以三色标记法是一个 false negative（假阴性）的算法。

三色标记法并发执行仍存在一个问题，即在 GC 过程中，对象指针发生了改变。比如下面的例子：

```
A（黑）-> B（灰）-> C（白）-> D（白）
```

正常情况下，D 对象最终会被标记为黑色，不应被回收。但在标记和用户程序并发执行过程中，用户程序删除了 C 对 D 的引用，而 A 获得了 D 的引用。标记继续进行，D 就没有机会被标记为黑色了（A 已经处理过，这一轮不会再被处理）。

```
A（黑）-> B（灰）-> C（白）
  ↓
 D（白）
```

为了解决这个问题，Go 使用了内存屏障技术，它是在用户程序读取对象、创建新对象以及更新对象指针时执行的一段代码，类似于一个钩子。垃圾收集器使用了写屏障（Write Barrier）技术，当对象新增或更新时，会将其着色为灰色。这样即使与用户程序并发执行，对象的引用发生改变时，垃圾收集器也能正确处理了。

一次完整的 GC 分为四个阶段：

- 1）标记准备(Mark Setup，需 STW)，打开写屏障(Write Barrier)
- 2）使用三色标记法标记（Marking, 并发）
- 3）标记结束(Mark Termination，需 STW)，关闭写屏障。
- 4）清理(Sweeping, 并发)

- 参考 fullstack

Q6 函数返回局部变量的指针是否安全？

答案

这在 Go 中是安全的，Go 编译器将会对每个局部变量进行逃逸分析。如果发现局部变量的作用域超出该函数，则不会将内存分配在栈上，而是分配在堆上。

Q7 非接口非接口的任意类型 T() 都能够调用 *T 的方法吗？反过来呢？

答案

- 一个 T 类型的值可以调用为*T 类型声明的方法，但是仅当此 T 的值是可寻址(addressable) 的情况下。编译器在调用指针属主方法前，会自动取此 T 值的地址。因为不是任何 T 值都是可寻址的，所以并非任何 T 值都能够调用为类型*T 声明的方法。
- 反过来，一个*T 类型的值可以调用为类型 T 声明的方法，这是因为解引用指针总是合法的。事实上，你可以认为对于每一个为类型 T 声明的方法，编译器都会为类型*T 自动隐式声明一个同名和同签名的方法。

哪些值是不可寻址的呢？

- 字符串中的字节；
- map 对象中的元素（slice 对象中的元素是可寻址的，slice 的底层是数组）；
- 常量；
- 包级别的函数等。

举一个例子，定义类型 T，并为类型 *T 声明一个方法 hello()，变量 t1 可以调用该方法，但是常量 t2 调用该方法时，会产生编译错误。

```
type T string
```

```go
func (t *T) hello() {
    fmt.Println("hello")
}

func main() {
    var t1 T = "ABC"
    t1.hello() // hello
    const t2 T = "ABC"
    t2.hello() // error: cannot call pointer method on t
}
```